

BASICS OF MECHATRONICS

MCT 231

FINAL REPORT



Name	Id
Salaheldin ahmed	24P0257
Yassin aly	24P0338
Mohamed adel rabie	24P0430
Khalid walled Kalboush	24P0034
Omar raafat gaber	24P0419

TABLE OF CONTENTS

Introduction	
Theme description	
Aim	
Product design specifications (PDS)	
Flowchart	
Block diagram.....	
Gantt chart.....	
Work breakdown structure (WBS).....	
Actuator sizing.....	
Codes	
Wiring diagrams AND PCB scheme.....	
Overview.....	

Introduction:

The arcade basketball game project is designed to simulate the excitement and competitiveness of real basketball in an interactive and automated system. Unlike traditional arcade setups, this project integrates mechanical design with embedded systems to create a smart, engaging experience. The system begins with a ball collection and lifting mechanism that transports balls from the input area to the processing stage. This is followed by a sorting mechanism that identifies specific ball characteristics—such as color—allowing for controlled gameplay logic and scoring variations. Finally, a shooting mechanism launches the ball toward a target or hoop, completing the gameplay cycle

Theme description:

The theme of our mechatronics project is inspired by classic arcade games, specifically the popular basketball game, where players aim to score points by shooting balls into hoops. Our project, titled “HOOPS MASTER” captures the fun, fast-paced spirit of arcade gameplay. The name reflects the satisfying motion and sound of a ball being shot and smoothly landing in its designated hoops. By integrating a lifting mechanism, a color-based sorting system, and a shooting module, “HOOPS MASTER” recreates the interactive challenge of arcade ball games in a compact, automated form – blending mechanical design, coding, and electronics to deliver both precision and entertainment

Our aim:

This project focuses on designing and building a mechatronics system capable of lifting, sorting, and shooting balls. The system uses various components such as an IR sensor, color sensor, servo motor, stepper motor, and an Arduino microcontroller.

The IR sensor detects the presence of the ball, while the color 3 sensor identifies the color of the ball. Once identified, the system uses the servo motor to sort the ball based on its color. After sorting, the stepper motor shoots the ball. The system is controlled by an Arduino, which processes the sensor data and sends commands to the motors. An LCD display shows the color of the detected ball. This project integrates hardware and software components to perform the tasks of detecting, sorting, and shooting with accuracy and efficiency

Product design specifications (PDS):

◆ 1. Function

The system shall:

- *Lift the ball to a specified height*
- *Identify the ball color*
- *Sort/direct the ball based on color*
- *Shoot the ball toward a designated target*
- *Calculate and display the score*

◆ 2. Performance Requirements

- *Small Ball detection response time*
- *Lifting time*
- *Color detection high accuracy*
- *System cycle time*
- *Product Life Span: Expected lifespan of the system is 3-5 years with proper maintenance. The servo motors and stepper motors are expected to last for at least 1,000 cycles, while the sensors may require calibration and cleaning after every 500 cycles.*

◆ 3. Physical Characteristics

- *Maximum dimensions: 100 x 100 cm*
- *Ball size: 20 mm*

◆ 4. Inputs and Outputs

Inputs:

- *Ball presence (IR/proximity sensor)*
- *Ball color (color sensor)*

Outputs:

- *Motor control signals*
- *Shooter activation*
- *Score display (LCD / 7-segment)*

◆ 5. Control System

- *Arduino*
- *Real-time processing of sensors*
- *Decision-making logic for sorting and shooting*
- *Score calculation algorithm*

◆ 6. Electrical Requirements

- *Supply voltage*
- *Power consumption*
- *Safe wiring and insulation required*

◆ 7. Mechanical Requirements

- *Stable lifting mechanism (conveyor)*
- *Reliable sorting mechanism (servo gate or different paths)*
- *Consistent shooting force (spring or motor-driven)*

◆ 8. Reliability

- *Replaceable components (motors, sensors)*

◆ 9. Cost Constraints

- *Total budget*
- *Use of available and low-cost components preferred*

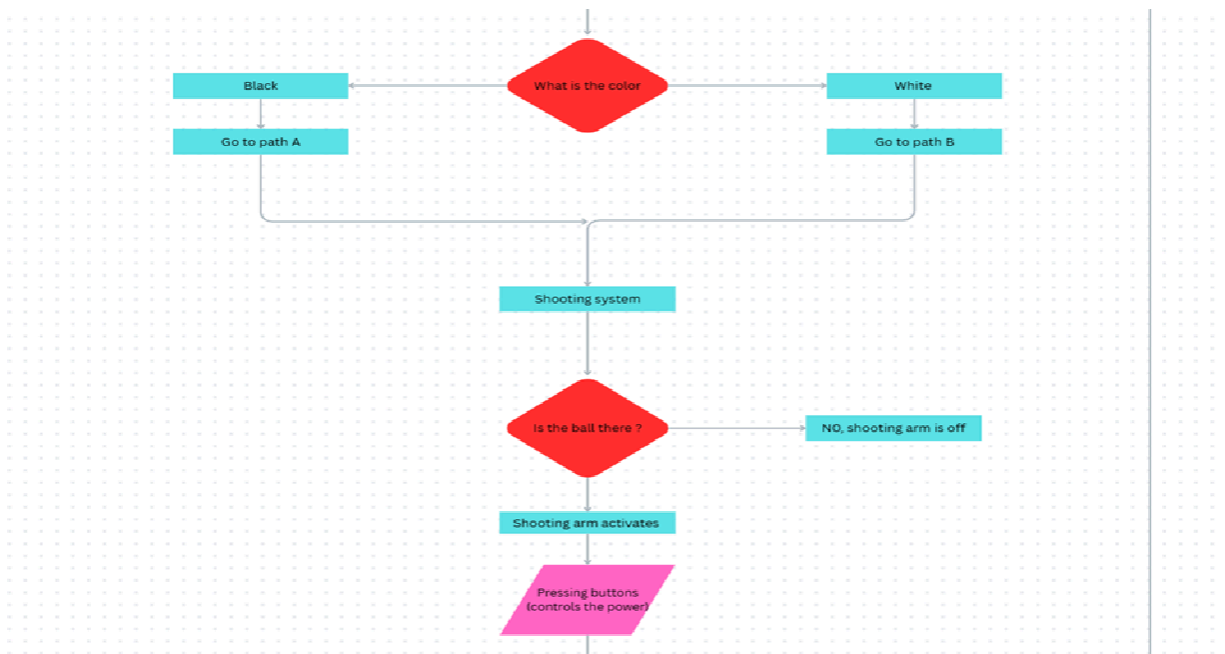
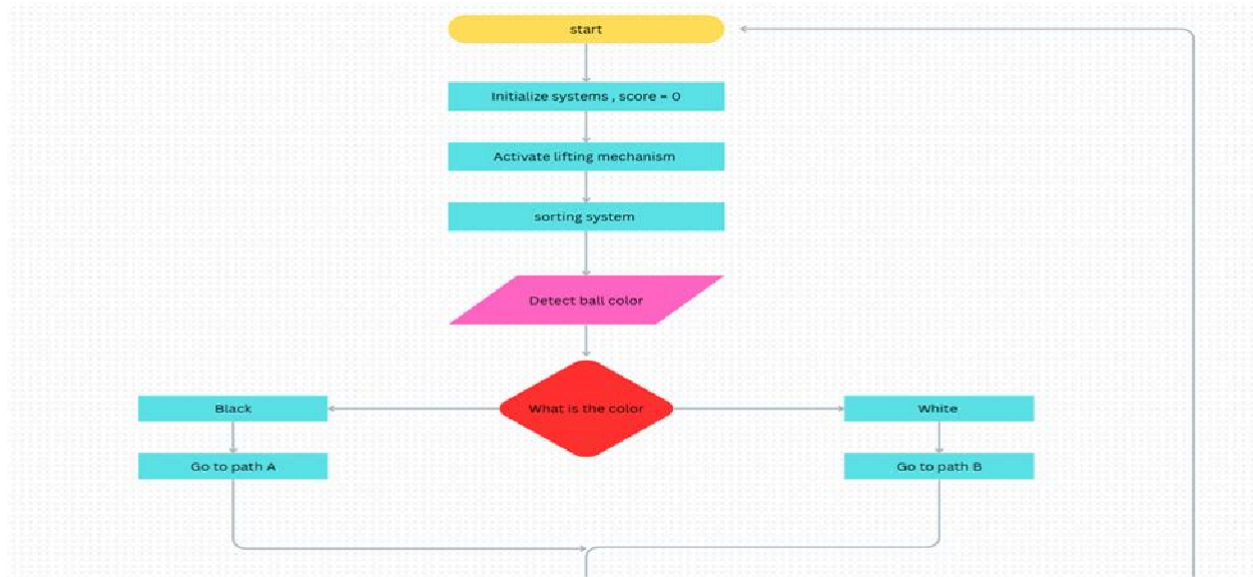
◆ 10. Testing

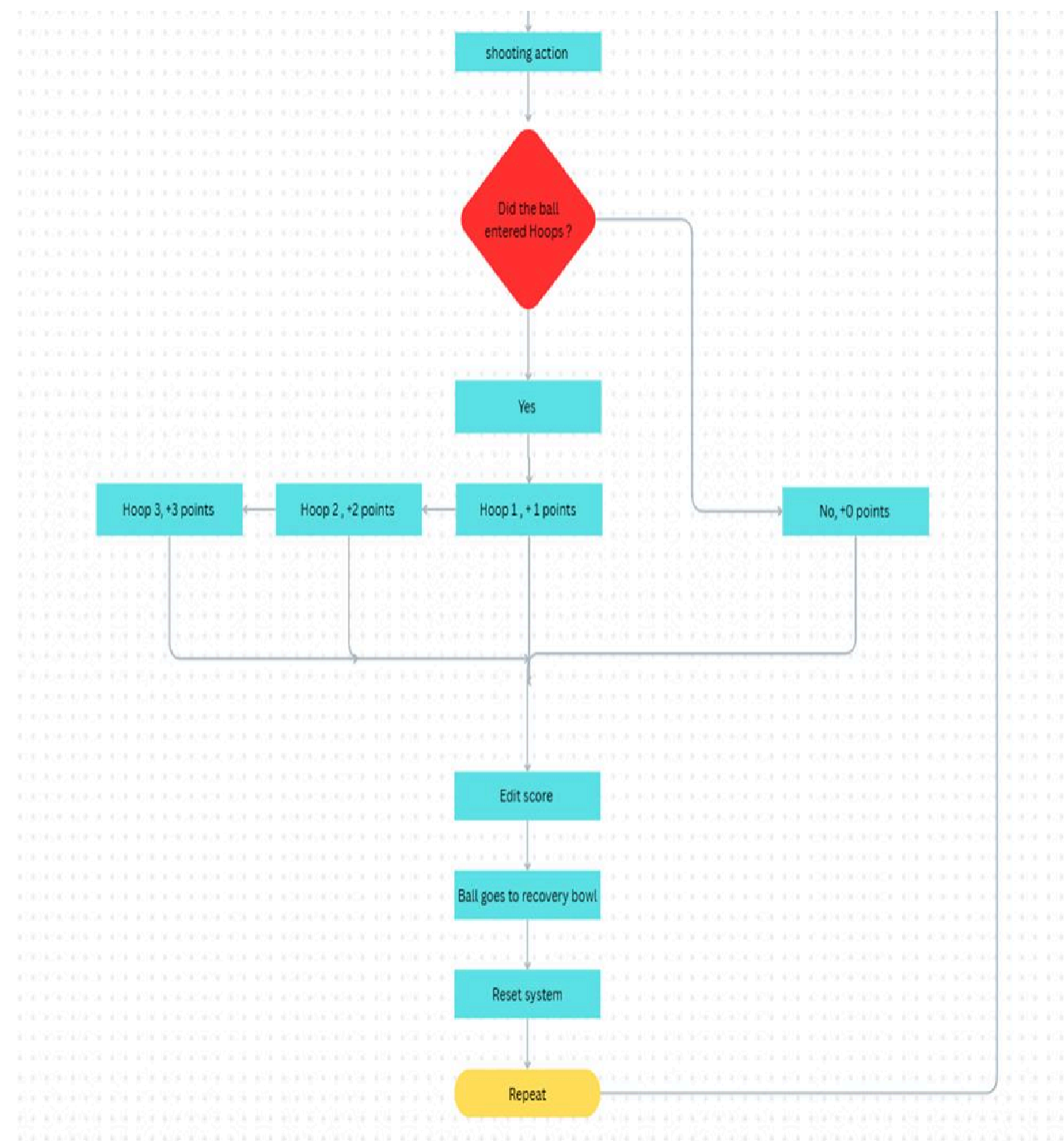
- *Sensor calibration tests*
- *Accuracy testing (color & shooting)*
- *System integration test*

◆ 11. Constraints

- *Use standard electronic components*
- *Comply with lab/project guidelines*

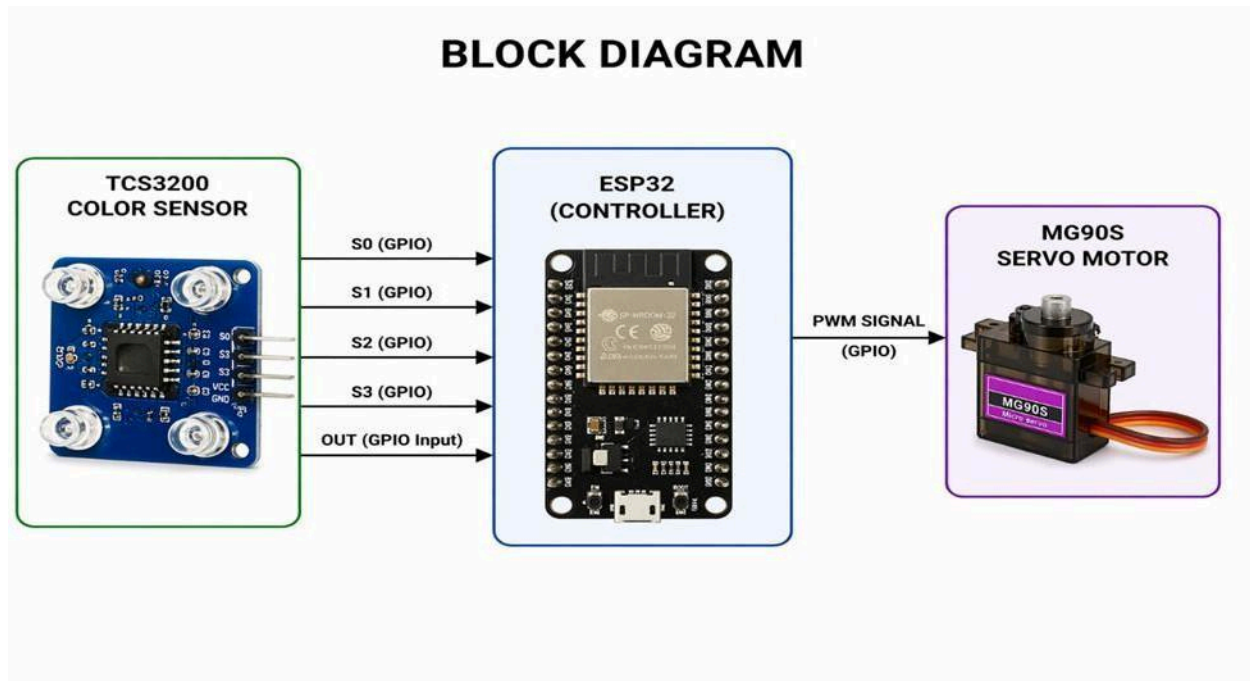
Close up on flowchart :



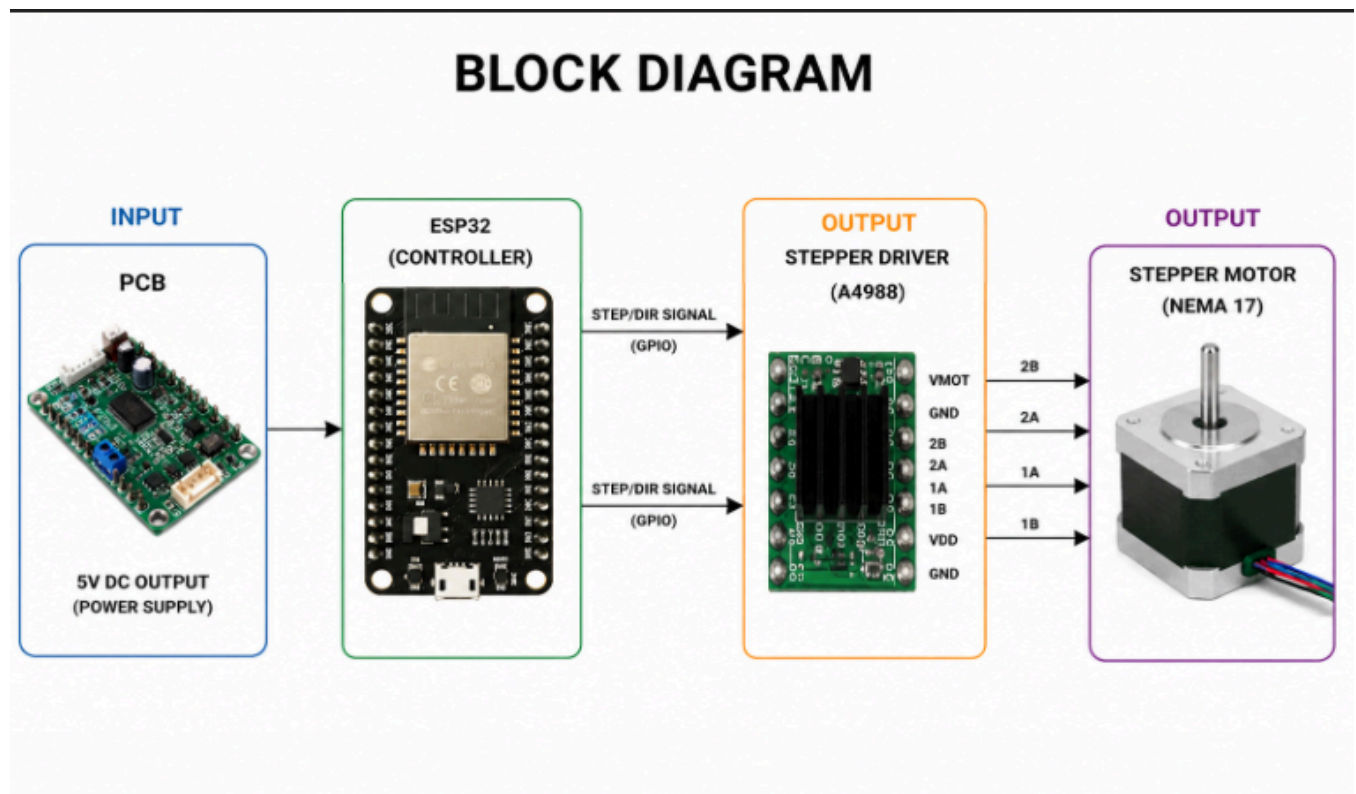


Block diagram:

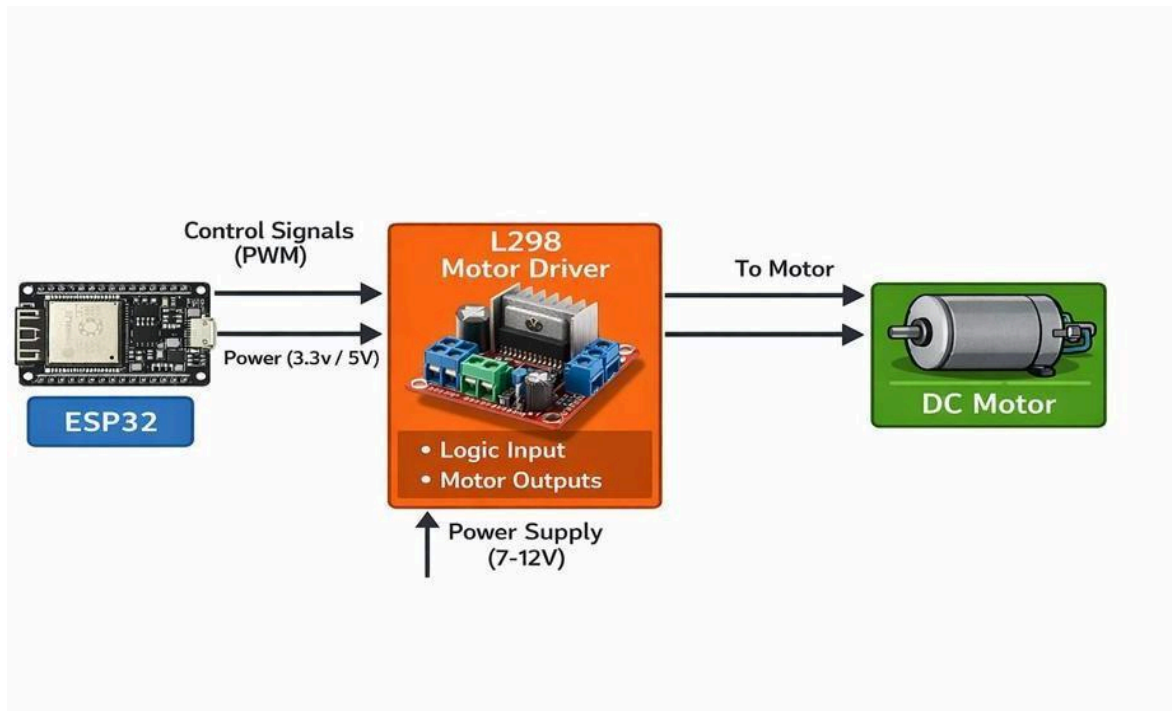
Sorting



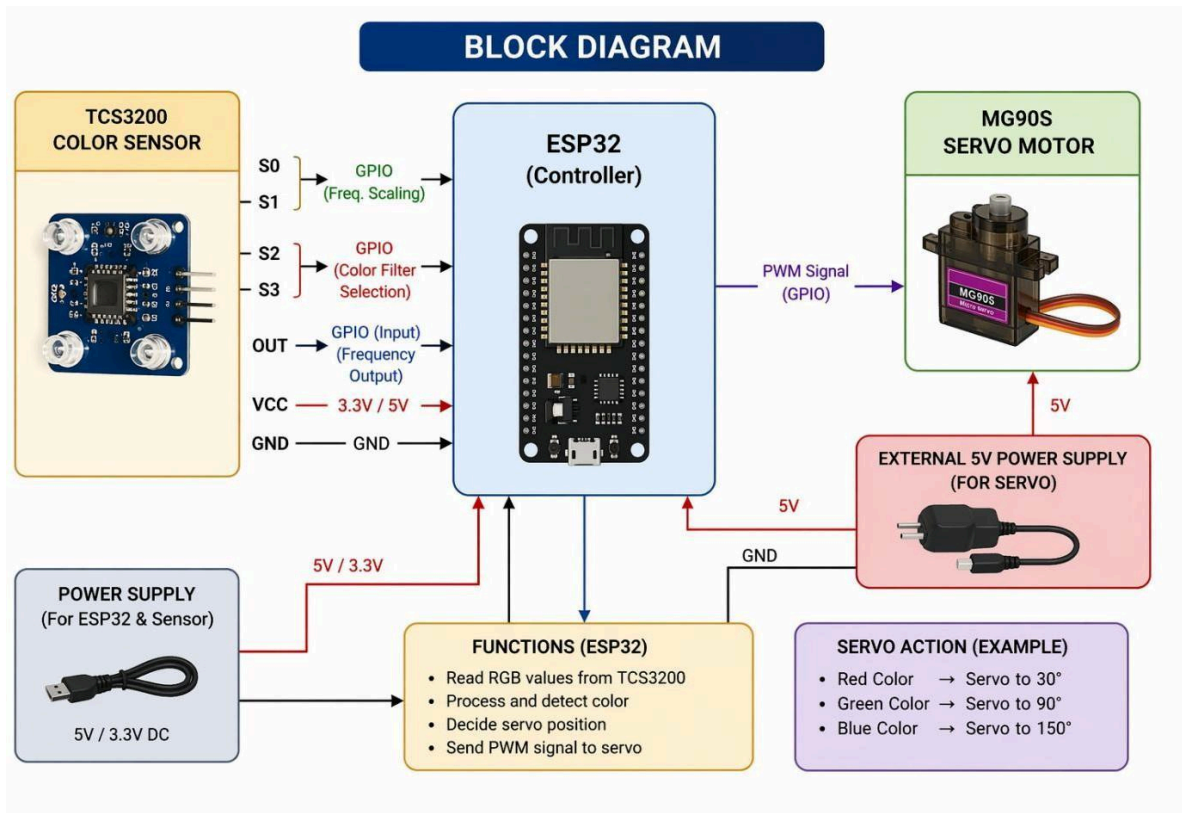
Shooting



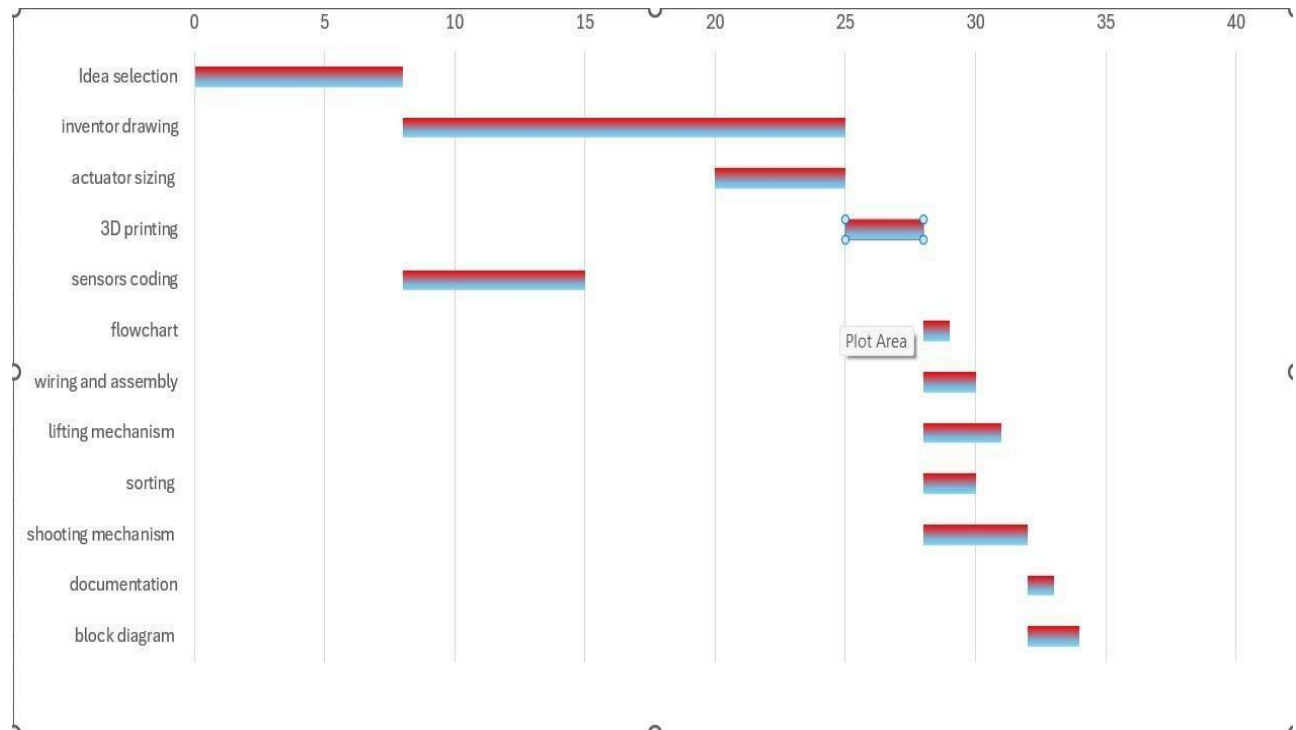
Lifting



Power block diagram:



Gantt chart:



Work breakdown structure (WBS):

1. System Design

- *1.1 Define requirements (ball size, colors, targets)*
- *1.2 Concept selection*
- *1.3 Component selection*

2. Mechanical System

- *2.1 Lifting mechanism design*
- *2.2 Ball guiding track*
- *2.3 Sorting mechanism (gates/ball paths)*
- *2.4 Shooting mechanism (shooting arm)*

3. Electrical & Electronics

- *3.1 Sensors*
 - *IR / proximity sensor (ball detection)*
 - *Color sensor*
- *3.2 Actuators*
 - *DC motors / servo motors*
 - *Shooting actuator*
- *3.3 arduino*
- *3.4 Power supply design*
- *3.5 Circuit wiring and PCB*

4. Control System

- *4.1 Motor control logic*
- *4.2 Color detection processing*
- *4.3 Decision-making logic (sorting + shooting)*
- *4.4 Scoring algorithm*
- *4.5 Display interface (LCD / 7-segment)*

5. Testing & Integration

- *5.1 Unit testing*
- *5.2 Integration testing*
- *5.3 Calibration (color sensor, shooting accuracy)*

6. Documentation & Presentation

- *C.1 Report writing*
- *C.2 Flowchart & diagrams*

Actuator sizing :

A) Lifting

1) Required Force

$$F = (m * g * \sin(\alpha)) + F_{\text{friction}}$$

$$*F = 0.325 * 9.81 * \sin(30^\circ) = 1.6 \text{ N}*$$

2) Friction Force

Safety margin (30%–50%):

$$*F_f \approx 2.2 \text{ N}*$$

3) Torque

$$T = F * r$$

$$*T = 2.2 * 0.02 = 0.044 \text{ Nm}*$$

4) Safety Factor

$$T_{\text{req}} \approx 0.1 \text{ Nm}$$

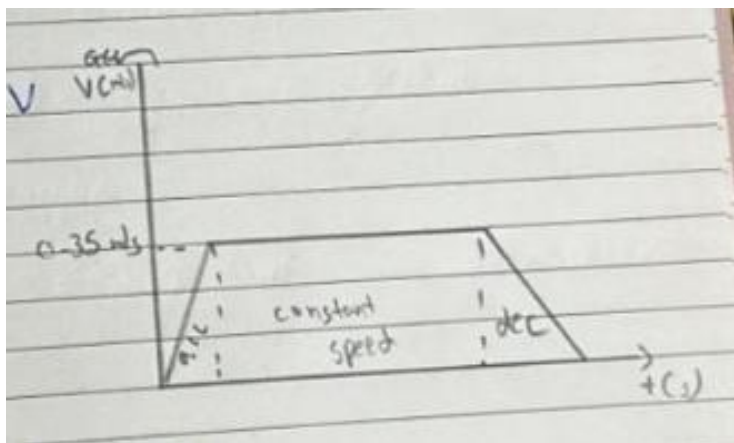
5) Motor

DC Motor $\rightarrow$ 12 V

6) Power

$$P = T * \omega$$

$$P = 0.1 * 3.456 = 0.35 \text{ W}$$



B) sorting

General Equation :

$$T_{\text{motor}} = T_{\text{resist}} + J_{\text{eq}} \cdot \alpha$$

1) Angular Acceleration

$$\alpha = (2 \cdot 0.785) / (1.5^2) = 0.7 \text{ rad/s}^2$$

2) Moment of Inertia

$$J = (1/3) \cdot m \cdot L^2$$

$$J = (1/3) \cdot 0.25 \cdot (0.12^2) = 0.0012 \text{ kg.m}^2$$

3) Inertia Torque

$$T_{\text{inertia}} = J \cdot \alpha$$

$$T = 0.0012 \cdot 0.7 = 0.0008 \text{ Nm}$$

4) Gravity Torque

$$r = L / 2 = 0.06 \text{ m}$$

$$T_g = m \cdot g \cdot r$$

$$T_g = 0.25 \cdot 9.81 \cdot 0.06 = 0.147 \text{ Nm}$$

5) Marble Torque

$$T = 0.005 \cdot 9.81 \cdot 0.06 = 0.003 \text{ Nm}$$

6) Friction Torque

$$T_{\text{friction}} \approx 0.03 \text{ Nm}$$

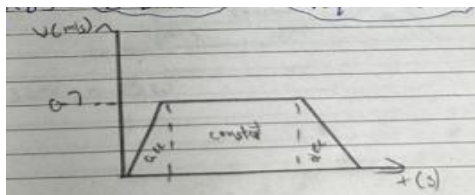
7) Total Resisting

$$\text{Torque } T_{\text{total}} = 0.147 +$$

$$0.05 \approx 0.2 \text{ Nm} \text{ With safety}$$

factor:

$$T_{\text{req}} = 0.4 \text{ Nm}$$



C) shooting

1. Moment of Inertia Calculation (J)

First, we calculate the total resistance to rotation (moment of inertia) of both the arm and the ball.

Arm Inertia (J_a)

For a rod rotating about one end:

$$J_a = \frac{1}{3} m_a L^2$$
$$J_a = \frac{1}{3} \times 0.008 \times (0.06)^2 = 9.6 \times 10^{-6} \text{ kg}\cdot\text{m}^2$$

Ball Inertia (J_b)

The ball is treated as a point mass at the strike radius:

$$J_b = m_p r^2$$
$$J_b = 0.005 \times (0.058)^2 = 1.68 \times 10^{-5} \text{ kg}\cdot\text{m}^2$$

Total Inertia (J_{total})

$$J_{\text{total}} = J_a + J_b = 2.64 \times 10^{-5} \text{ kg}\cdot\text{m}^2$$

2. Required Torque Calculation

To achieve a ball exit velocity of **5 m/s** with a radius of **0.058 m**, the required angular velocity is:

$$\omega = \frac{v}{r} = \frac{5}{0.058} \approx 86.2 \text{ rad/s}$$

Angular Acceleration (α)

Assuming the motion occurs over a 90° ($\pi/2$ radians) rotation:

$$\alpha = \frac{\omega^2}{2\Delta\theta} = \frac{(86.2)^2}{\pi} \approx 2365 \text{ rad/s}^2$$

Acceleration Torque

$$T_{\text{accel}} = J \cdot \alpha$$

$$T_{\text{accel}} = (2.64 \times 10^{-5}) \times 2365 = 0.0625 \text{ N}\cdot\text{m}$$

Gravity Torque (30° Incline)

$$T_g = (m_p \cdot r + m_a \cdot \frac{L}{2}) \cdot g \cdot \sin(30^\circ)$$

$$T_g = 0.0026 \text{ N}\cdot\text{m}$$

Total Required Torque

$$T_{total} = T_{accel} + T_g = 0.0625 + 0.0026 = 0.0651 \text{ N}\cdot\text{m}$$

Projectile calculations :

Setup:

- Launch angle: 30° (from ramp, fixed)
- Arm length: $6.4 \text{ cm} = 0.064 \text{ m}$
- Gravity: $g = 9.81 \text{ m/s}^2$

Step 1 — Launch speed calculation

Tip speed of arm = angular velocity $\times$ arm length

$$v = \omega \times r$$

Where:

- MGGG6R speed at 6V: $0.17 \text{ sec} / 60^\circ$
- $\omega = (60^\circ / 0.17\text{s}) \times (\pi/180) = 6.16 \text{ rad/s}$
- $r = 0.064 \text{ m}$

$$v = 6.16 \times 0.064 = 0.394 \text{ m/s}$$

$$v_x = v \times \cos(30^\circ) = 0.394 \times 0.866 = 0.341$$

$$\text{m/s } v_y = v \times \sin(30^\circ) = 0.394 \times 0.5 =$$

$$0.197 \text{ m/s}$$

$$\text{Range} = (v^2 \times \sin(2\theta)) / g$$

$$\text{Range} = (0.394^2 \times \sin(60^\circ)) /$$

$$9.81 \text{ Range} = (0.155 \times 0.866) /$$

$$9.81$$

$$\text{Range} = 0.0136 \text{ m} = 1.37$$

$$\text{cm } H_{\text{max}} = (v^2 \times \sin^2\theta) /$$

$$(2g) H_{\text{max}} = (0.155 \times 0.25)$$

$$/ 19.62$$

$$H_{\text{max}} = 0.00197 \text{ m} = 0.2 \text{ cm}$$

$$t = (2 \times v \times \sin \theta) / g$$

$$t = (2 \times 0.3G4 \times 0.5) /$$

$$G.81 \quad t = 0.0401 \text{ s} = 40$$

ms

$$KE = \frac{1}{2} \times m \times v^2$$

$$KE = \frac{1}{2} \times 0.005 \times$$

$$0.3G4^2 \quad KE = 3.88 \times 10^{-4}$$

J

$$KE = 0.388 \text{ mJ}$$

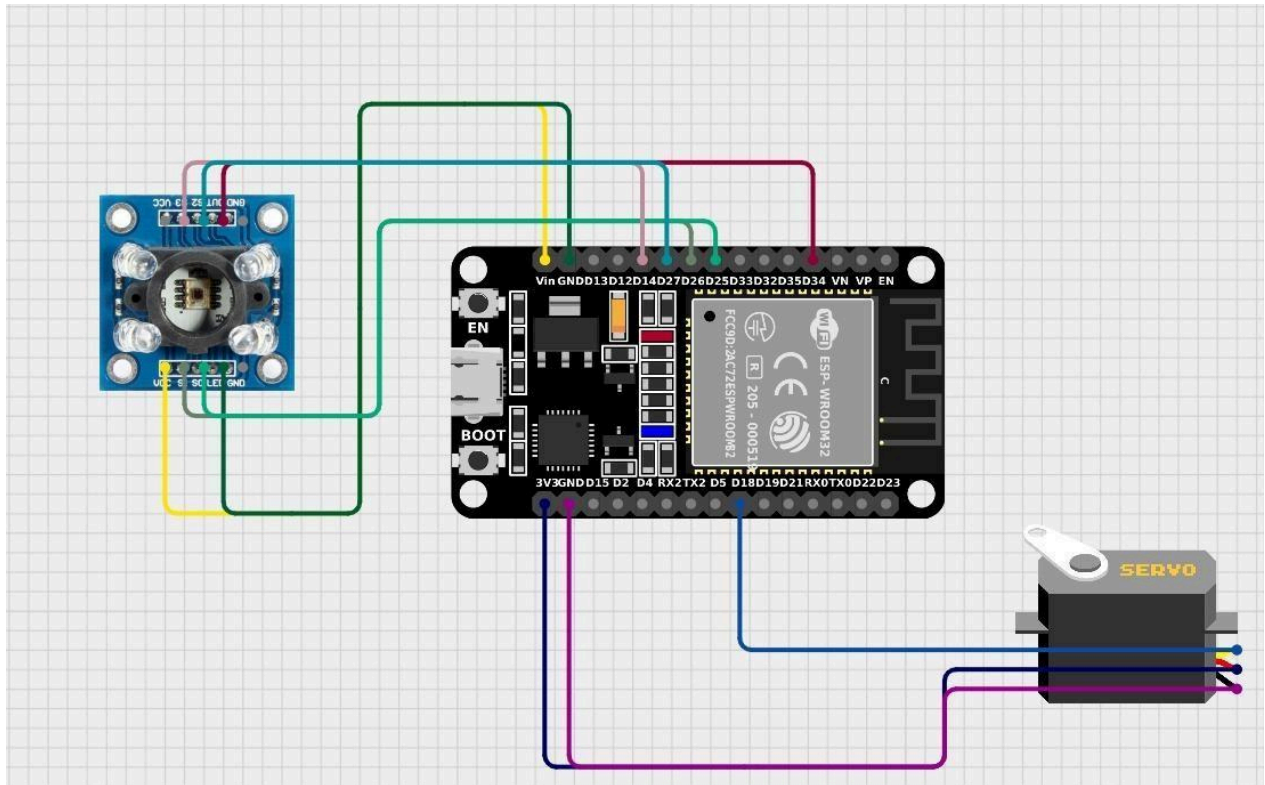
$$p = m \times v$$

$$p = 0.005 \times 0.3G4$$

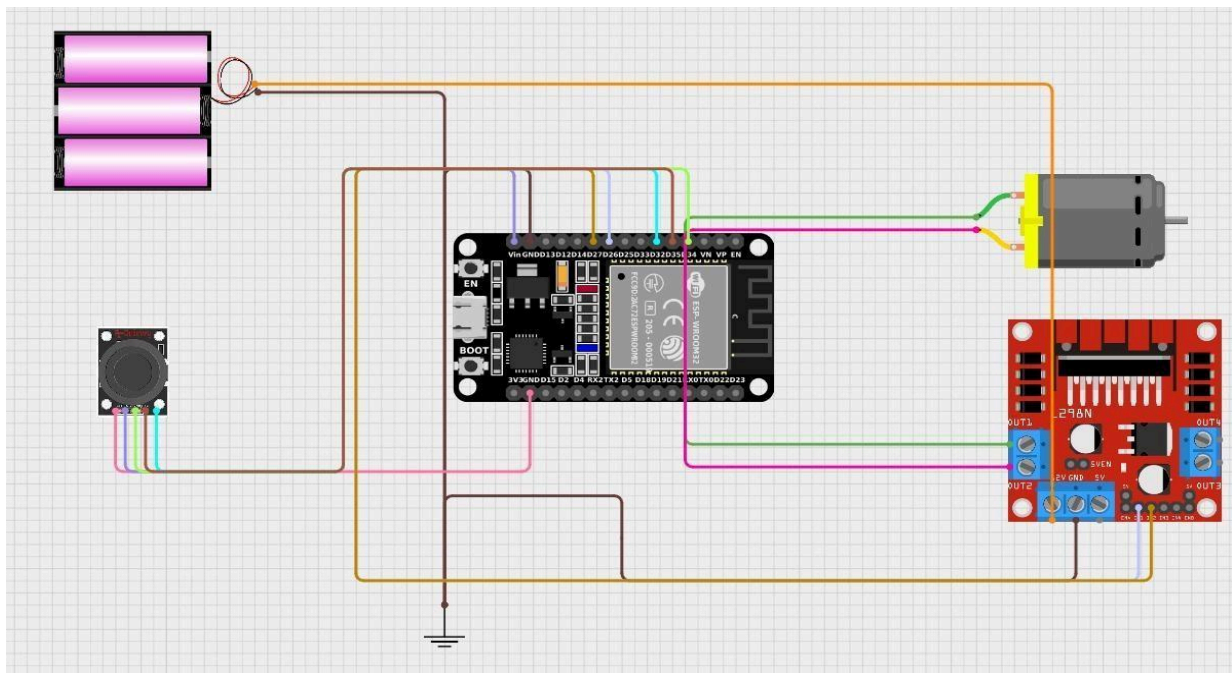
$$p = 1.67 \times 10^{-3} \text{ kg} \cdot \text{m/s}$$

Wiring diagrams s PCB schemes:

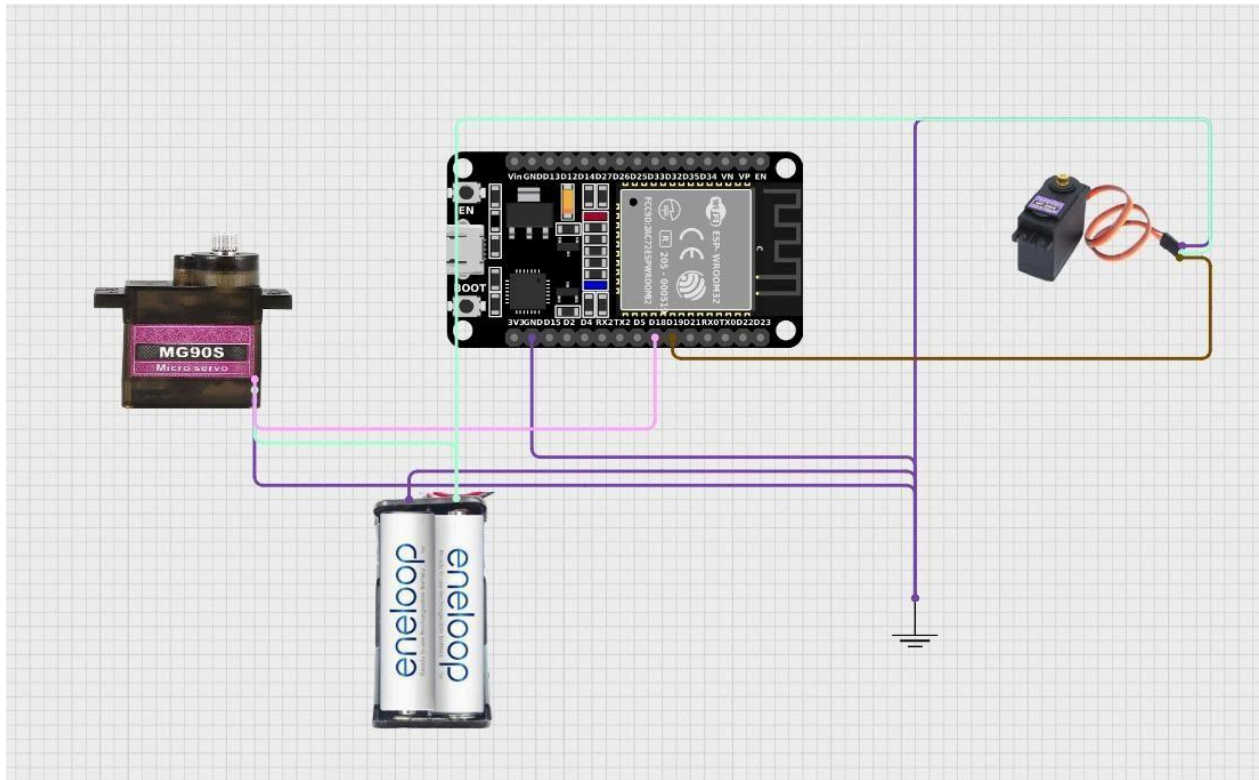
Sorting



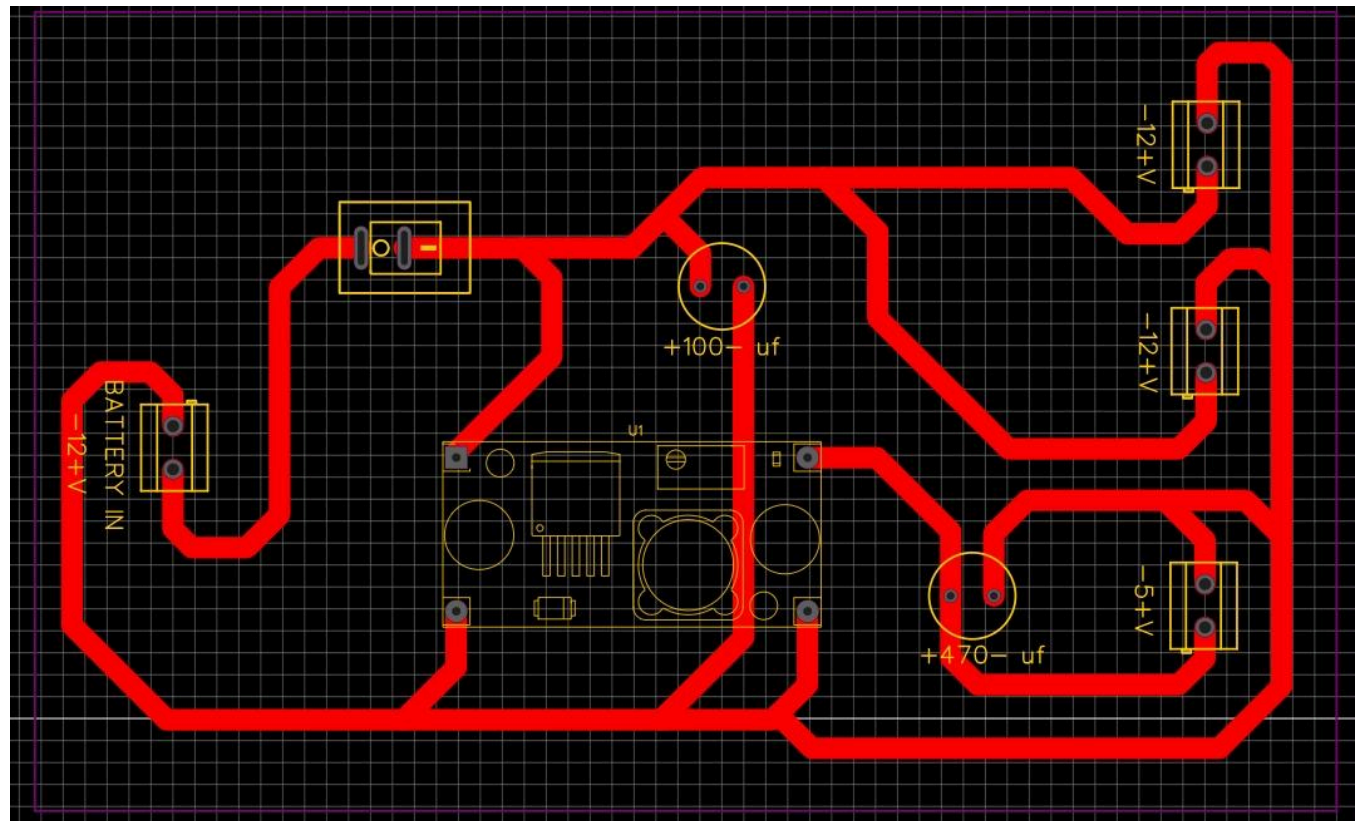
Lifting



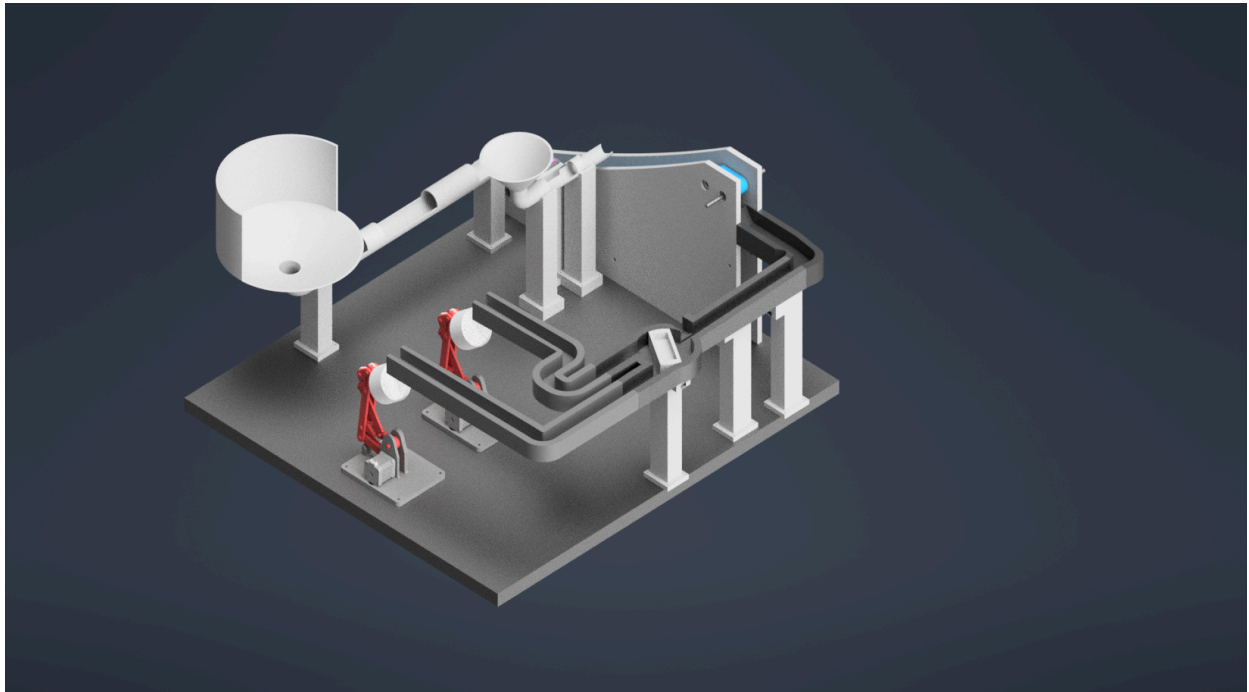
Shooting



PCB :



3D RENDER for the full integration:



Arduino codes:

Shooting

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
#include
```

```
<ESP32Servo.h>
```

```
WebServer
```

```
server(80); Servo
```

```
shootServo; Servo
```

```
gateServo;
```

```
int targetAngle = 45;
```

```
const int SHOOT_REST =
```

```
35; const int GATE_REST =
```

```
0; const int GATE_OPEN =
```

```
90;
```

```
void setup() {
```

```
  shootServo.attach(18, 500,
```

```
  2400);
```

```
  gateServo.attach(19, 500, 2400);
```

```
  shootServo.write(SHOOT_REST);
```

```
  gateServo.write(GATE_REST);
```

```
  delay(1000);
```

```
  WiFi.softAP("esp32shooting", "12345678");
```

```
server.on("/", []) {
```

```

String p = "<!DOCTYPE html><html><head><meta
name='viewport' content='width=device-width,initial-scale=1'>";

p += "<style>";

p += "*{box-sizing:border-box;margin:0;padding:0}";

p += "body{background:radial-gradient(ellipse at top,#1a1a3e
0%,#0a0a1e
70%);color:#fff;font-family:sans-serif;min-height:100vh;display:flex;align-
items:center;justify-content:center;padding:20px;overflow:hidden}";

p += ".bg-glow{position:fixed;width:600px;height:600px;background:radial-
gradient(circle,rgba(255,0,80,0.2) 0%,transparent 70%);top:-200px;right:-200px;pointer-
events:none;animation:pulse 4s ease-in-out infinite}";

p +=
".bg-glow2{position:fixed;width:500px;height:500px;background:radial-
gradient(circle,rgba(0,200,255,0.2)
0%,transparent
70%);bottom:-150px;left:- 150px;pointer-events:none;animation:pulse 4s
ease-in-out infinite 2s}";

p += ".bg-glow3{position:fixed;width:400px;height:400px;background:radial-
gradient(circle,rgba(150,0,255,0.15) 0%,transparent 70%);top:40%;left:30%;pointer-
events:none;animation:pulse 6s ease-in-out infinite 1s}";

p += "@keyframes
pulse{0%,100%{opacity:0.5;transform:scale(1)}50%{opacity:1;transform:scale(1.15)}}";

p += ".card{position:relative;background:rgba(15,15,35,0.8);backdrop-
filter:blur(25px);border:1px solid
rgba(255,255,255,0.15);border-radius:32px;padding:50px
38px;width:100%;max-width:440px;box-shadow:0 30px 80px rgba(0,0,0,0.8),inset 0
1px 0 rgba(255,255,255,0.1);z-index:1}";

p += ".card::before{content:'';position:absolute;inset:-1px;border-
radius:32px;padding:1px;background:linear-
gradient(135deg,rgba(0,212,255,0.3),transparent,rgba(255,51,102,0.3));-web
kit-mask:linear-gradient(#fff 0 0) content-box,linear-gradient(#fff 0
0);-webkit-mask-
composite:xor;mask-composite:exclude;pointer-events:none}";

p += ".dot{display:inline-block;width:10px;height:10px;border-
radius:50%;background:#00ff88;box-shadow:0 0 15px #00ff88;margin:0
8px;vertical-align:middle;animation:blink 2s ease infinite}";

p += "@keyframes blink{0%,100%{opacity:1}50%{opacity:0.3}}";

p +=
".subtitle{text-align:center;color:#888;font-size:12px;letter-spacing:3px;text-

```



```
transform:uppercase;margin-bottom:12px}";
```

```

p += "h1{text-align:center;font-size:34px;font-weight:900;margin-
bottom:35px;background:linear-
gradient(90deg,#ff3366,#ff66aa,#00d4ff,#00eaff);background-size:300%
auto;-webkit- background-clip:text;-webkit-text-fill-color:transparent;letter-
spacing:2px;animation:gradient 4s ease infinite}";

p += "@keyframes gradient{0%,100%{background-position:0%
center}50%{background- position:100% center}}";

p += ".angle-box{background:linear-
gradient(135deg,rgba(0,0,0,0.5),rgba(0,30,60,0.3));border:1px solid
rgba(0,212,255,0.4);border-radius:22px;padding:28px;margin-bottom:28px;t
ext-align:center;position:relative;overflow:hidden;box-shadow:inset 0 0 30px
rgba(0,212,255,0.1)}}";

p += ".angle-box::before{content:'';position:absolute;top:0;left:-
100%;width:100%;height:2px;background:linear-
gradient(90deg,transparent,#00d4ff,transparent);animation:scan 3s linear
infinite}";

p += ".angle-box::after{content:'';position:absolute;bottom:0;right:-
100%;width:100%;height:2px;background:linear-
gradient(90deg,transparent,#ff3366,transparent);animation:scan2 3s linear infinite
1.5s}";

p += "@keyframes scan{to{left:100%}}";

p += "@keyframes scan2{to{right:100%}}";

p += ".angle{font-size:88px;font-weight:900;color:#00d4ff;text-shadow:0 0 40px
rgba(0,212,255,0.9),0 0 80px
rgba(0,212,255,0.4);font-family:monospace;letter-spacing:- 3px;line-height:1}";
p +=

".deg{font-size:40px;color:#666;vertical-align:top;margin-left:5px}";

p += ".label-small{font-size:11px;color:#888;letter-spacing:3px;text-
transform:uppercase;margin-top:8px}";

p += ".slider-wrap{position:relative;margin:25px 0}";

p +=
{width:100%;height:12px;border-radius:6px;background:linear-
gradient(90deg,rgba(0,212,255,0.3) 0%,rgba(255,51,102,0.3)
100%);outline:none;-webkit- appearance:none}";

p += "input[type=range]::-webkit-slider-thumb{-webkit-
appearance:none;width:38px;height:38px;border-radius:50%;background:radial-
gradient(circle,#fff 20%,#00eaff 50%,#0088cc 100%);cursor:pointer;box-shadow:0 0

```

30px

```
rgba(0,212,255,1),0 0 60px rgba(0,212,255,0.6),inset 0 0 10px  
rgba(255,255,255,0.5);border:2px solid rgba(255,255,255,0.8)}}";
```

```
p += "input[type=range]::-moz-range-thumb{width:38px;height:38px;border-  
radius:50%;background:radial-gradient(circle,#fff,#00eaff);cursor:pointer;border:2px  
solid #fff}";
```

```
p +=  
".labels{display:flex;justify-content:space-between;color:#666;font-  
size:12px;margin-bottom:35px;margin-top:10px;letter-spacing:1px}";
```

```
p += ".labels span{color:#aaa;font-weight:600}";
```

```
p += "button{width:100%;padding:32px;font-size:32px;font-  
weight:900;color:#fff;background:linear-gradient(135deg,#ff3366 0%,#ff0044  
50%,#aa0033 100%);border:none;border-radius:20px;cursor:pointer;box-shadow:0  
15px 40px rgba(255,51,102,0.6),inset 0 1px 0 rgba(255,255,255,0.25),inset 0 -3px 0  
rgba(0,0,0,0.2);letter-spacing:5px;position:relative;overflow:hidden;transition:all  
0.15s;text-shadow:0 2px 10px rgba(0,0,0,0.5)}";
```

```
p +=  
"button::before{content:'';position:absolute;top:50%;left:50%;width:0;height:0;bor-  
der-radius:50%;background:rgba(255,255,255,0.4);transform:translate(-50%,-  
50%);transition:width 0.5s,height 0.5s}";
```

```
p += "button:active{transform:scale(0.95);box-shadow:0 5px 20px  
rgba(255,51,102,0.8)}";
```

```
p += "button:active::before{width:500px;height:500px}";
```

```
p += ".status{text-align:center;margin-top:25px;color:#00ff88;font-size:14px;letter-  
spacing:4px;font-weight:700;min-height:20px;text-shadow:0 0 10px  
rgba(0,255,136,0.5)}";
```

```
p += ".firing{color:#ff3366!important;text-shadow:0 0 15px  
rgba(255,51,102,0.8)!important;animation:flash 0.25s ease infinite}";
```

```
p += "@keyframes  
flash{0%,100%{opacity:1;transform:scale(1)}50%{opacity:0.5;transform:scale(1.05)}}";
```

```
p += ".footer{text-align:center;margin-top:18px;color:#444;font-size:10px;letter-  
spacing:2px}";
```

```
p += "</style></head><body>";
```

```
p += "<div class='bg-glow'></div><div class='bg-glow2'></div><div class='bg-  
glow3'></div>";
```

```
p += "<div class='card'>";
```

```

    p += "<div class='subtitle'><span class='dot'></span>System Armed<span  

class='dot'></span></div>";

    p += "<h1>SHOOTER CONTROL</h1>";

    p += "<div class='angle-box'>";

    p += "<div class='angle'><span id='v'>45</span><span  

class='deg'>Cdeg;</span></div>";

    p += "<div class='label-small'>Target  

Angle</div>"; p += "</div>";

    p += "<div class='slider-wrap'>";

    p += "<input type='range' min='45' max='155' value='45'  

oninput=\"document.getElementById('v').innerText=this.value;fetch('/s?a='+this.value)\"  

\">\"  

;

    p += "</div>";

    p += "<div class='labels'><span>MIN 45Cdeg;</span><span>MAX  

155Cdeg;</span></div>";

    p += "<button onclick=\"fetch('/shoot');var  

s=document.getElementById('st');s.innerText='FIRING';s.classList.add('firing');setTimeout(  

function(){s.innerText='GATE  

OPEN';setTimeout(function(){s.innerText='READY';s.classList.remove('firing')},1000)},600)\"  

\">FIRE</button>";

    p += "<div class='status' id='st'>READY</div>";

    p += "<div class='footer'>MG996R Cbull; MG90S Cbull;  

ESP32</div>"; p += "</div></body></html>";

    server.send(200, "text/html", p);

});

server.on("/s", []() {
    targetAngle =
    server.arg("a").toInt();
    server.send(200, "text/plain",
    "ok");
});

```

```
server.on("/shoot", []() {  
  server.send(200, "text/plain", "ok");
```

```
  
  // Snap shooter to target  
  angle  
  shootServo.write(targetAngle  
); delay(300);
```

```
  
  // Return shooter to rest  
  shootServo.write(SHOOT_REST);  
  delay(300);
```

```
  
  // Open gate 90° clockwise  
  gateServo.write(GATE_OPEN);  
  delay(1000);
```

```
  
  //      Close      gate  
  gateServo.write(GATE_REST  
);  
});
```

```
  
server.begin();  
}
```

```
  
void loop() {  
  server.handleClient();  
}
```

Sorting

```
#include <ESP32Servo.h>
```

```
Servo myServo;
```

```
//----- SERVO -----
```

```
int servoPin = 18;
```

```
int defaultPos = 90;
```

```
int rightPos = 145; // white
```

```
int leftPos = 45; // black
```

```
//----- TCS3200 PINS-----
```

```
int S0 =
```

```
25; int S1
```

```
= 26; int
```

```
S2 = 27;
```

```
int S3 =
```

```
14;
```

```
int sensorOut = 34;
```

```
//----- THRESHOLDS -----
```

```
int detectChange = 40; // sensitivity for ball presence
```

```
int colorThreshold = 150; // avg > this = black, else  
white
```

```
//----- EMPTY BASELINE-----
```

```
int emptyR =
```

```
0; int emptyG
```

```
= 0; int
```

```
emptyB = 0;
```



```
//----- READ FUNCTIONS-----
```

```
int readRed() {  
    digitalWrite(S2,  
        LOW);  
    digitalWrite(S3,  
        LOW);  
    return pulseIn(sensorOut, LOW);  
}
```

```
int readGreen() {  
    digitalWrite(S2,  
        HIGH);  
    digitalWrite(S3,  
        HIGH);  
    return pulseIn(sensorOut, LOW);  
}
```

```
int readBlue() {  
    digitalWrite(S2,  
        LOW);  
    digitalWrite(S3,  
        HIGH);  
    return pulseIn(sensorOut, LOW);  
}
```

```
//----- SETUP-----
```

```
void setup() {  
    Serial.begin(115200);
```

```
myServo.attach(servoPin, 500, 2400);
```

```
myServo.write(defaultPos);
```

```
pinMode(S0, OUTPUT);
```

```
pinMode(S1, OUTPUT);  
pinMode(S2, OUTPUT);  
pinMode(S3, OUTPUT);  
pinMode(sensorOut, INPUT);
```

```
digitalWrite(S0,  
HIGH);  
digitalWrite(S1,  
LOW);
```

```
Serial.println("Keep path empty... calibrating");
```

```
delay(2000); // leave chute empty
```

```
// Save empty-space readings  
emptyR = readRed();  
emptyG =  
readGreen(); emptyB  
= readBlue();
```

```
Serial.println("Ready.");  
}
```

```
//----- LOOP -----  
void loop() {
```

```
int redValue = readRed();  
int greenValue = readGreen();  
int blueValue = readBlue();
```

```
int total = redValue + greenValue +  
blueValue; int avg = total / 3;
```

```
Serial.print("R: "); Serial.print(redValue);  
Serial.print(" G: ");  
Serial.print(greenValue); Serial.print(" B:  
"); Serial.print(blueValue); Serial.print("  
TOTAL: "); Serial.print(total);  
Serial.print(" AVG: "); Serial.println(avg);
```

```
// ----- NO BALL DETECTED -----  
// If readings are close to empty baseline, do not  
move if (abs(redValue - emptyR) < detectChange CC  
abs(greenValue - emptyG) < detectChange  
CC abs(blueValue - emptyB) <  
detectChange) {
```

```
Serial.println("NO BALL -  
WAITING"); delay(100);  
return;  
}
```

```
// ----- BLACK -----  
if (avg > colorThreshold) {  
Serial.println("BLACK -> Left  
45");  
  
myServo.write(leftPos)  
; delay(600);
```

```
myServo.write(defaultPos);  
delay(1500);  
}  
  
// ----- WHITE -----  
else {  
  Serial.println("WHITE -> Right 45");  
  
  myServo.write(rightPos);  
  delay(600);  
  
  myServo.write(defaultPos);  
  delay(1500);  
}  
}
```

Lifting

```
const int IN1 = 27;
```

```
const int IN2 = 26;
```

```
const int ENA = 14;
```

```
const int VRY = 35;
```

```
const int SW = 32;
```

```
bool running = true;
```

```
void setup() {  
  pinMode(IN1, OUTPUT);  
  pinMode(IN2, OUTPUT);  
  pinMode(SW, INPUT_PULLUP);  
  ledcAttach(ENA, 1000, 8);  
}
```

```
void loop() {  
  // Button press toggles  
  stop/start if (digitalRead(SW)  
  == LOW) { running = !running;  
    delay(300); // debounce  
}
```

```
if (!running) {  
  // Stopped  
  digitalWrite(IN1, LOW);  
  digitalWrite(IN2, LOW);
```

```
    ledcWrite(ENA, 0);  
    return;  
}  
  
int joy = analogRead(VRY);
```

```
if (joy < 1500) {  
    // Forward →  
    clockwise  
    digitalWrite(IN1,  
    HIGH);  
    digitalWrite(IN2,  
    LOW); ledcWrite(ENA,  
    255);  
  
} else if (joy > 2500) {  
    // Backward → anticlockwise  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    ledcWrite(ENA, 255);  
  
} else {  
    // Center → stop  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);  
    ledcWrite(ENA, 0);  
}  
}
```

Conclusion:

The use of advanced sensors and actuators ensured precision in gameplay, providing players with real-time feedback on their performance. This integration of mechatronic principles not only enhanced the overall gaming experience but also demonstrated the practical application of theoretical concepts learned throughout the project.

As players competed for high scores, the dynamic interactions captured a spirit of friendly competition and community. Observers cheered, and participants celebrated each successful shot, underscoring how technology can bring people together in enjoyment and shared excitement.

Looking ahead, the project serves as a stepping stone for future advancements.

Enhancements such as improved user interfaces, AI-driven gameplay analytics, and even virtual reality integration could elevate the arcade basketball experience further.

Overall, this mechatronics project has successfully created an engaging platform that marries gameplay with technology, inspiring players to not only enjoy basketball but also appreciate the intricate workings behind the scenes. As the arcade prepares for future iterations of this game, we are reminded of the endless possibilities that lie at the intersection of sports and technology. Let the next round of play begin